



Computer Games Development CW208

GDD

Year IV

[Jake Fitzgerald]
[C00288105]

[Student Name]
[Student Number]

[13/04/2026]

[Declaration form to be attached]

Contents

Acknowledgements.....	4
Game Overview.....	3
- Genre and Platform.....	3
- Concept.....	3
- Design Goals.....	3
Gameplay.....	4
- Core Loop.....	4
- Input.....	4
- Note Visuals.....	5
- Scoring.....	5
- Game Modes.....	6
- Statistics.....	6
Songs.....	7
- Song List.....	7
- Custom Midi Songs.....	8
References.....	8

Acknowledgements

Thank you to my supervisor Amr Abdelhafez for the constant feedback and support offered during the development of this project. I also thank the Carlow SETU for providing the resources available to assist me during the project.

Game Overview

BeatBreaker is a single-player, PC-based piano learning game. The player selects a song from a MIDI library, then plays along by pressing the correct piano keys as colour-coded note fall down the screen towards a visual keyboard. The genre is a rhythm game similar to games like Synthesia or Guitar Hero.

The game is built entirely in C++ using the SFML 3.0 graphics and audio library. It supports physical MIDI controller input via the Windows MIDI API, and connects to an Azure SQL Server database that hosts an online leaderboard with other user's scores and their performances which can be downloaded in the Midi format, which can be viewed in the game.

Genre and Platform

Genre – Rhythm game with educational goals (learning to play a song on the piano)

Platform – Windows (requires the Windows Multimedia Library)

Target Audience – People that want to improve their piano skills (beginner to intermediate)

Design Goals

End goals for user – Learn to play a specific song on piano by focusing on these things:

- Make piano learning approachable by wrapping it in familiar rhythm-game mechanics.
- Reward accuracy over speed the scoring system distinguishes perfect timing from sloppy hits.
- Let players hear themselves improve through live statistics, personal bests, and rank grades.
- Support self-directed practice via Practice Mode, Preview Mode, and Ghost Mode.
- Allow players to share and compare performances through MIDI upload and leaderboard downloads (encourages a competitive community with other users).

Gameplay

Every session in Beat Breaker follows this loop:

- The player navigates from the Main Menu to Song Select and picks a song and difficulty.
- The game parses the chosen MIDI file and converts all notes into falling notes used in the gameplay scene.
- The song starts and notes fall toward the keyboard at the bottom of the screen. Notes are spawned the viewport based on their start time and BPM.
- The player presses keys (mouse click, keyboard, or MIDI controller). Each keypress is checked against the three hit windows (early, perfect, late) attached to the corresponding falling note.
- Score and live statistics update instantly on screen. Notes change colour to reflect the hit quality. Missed notes are removed when they pass the kill trigger below the keyboard.
- When all notes have passed then the song ends and the game transitions to the Results screen. From there the player can save their performance as a MIDI file or submit their score to the leaderboard which also uploads their Midi to the server's database.

Input

BeatBreaker supports three input types:

Mouse click

Players can click on a key or drum piece during gameplay or any visualiser to hear the sound of the instrument and it's corresponding pitch.

Computer keyboard

Used for typing into the input fields for saving and loading different types of data.

(Midi file save/load, server username save)

MIDI controller (Midi Piano)

Used to detect the keys/drum pads pressed on a Midi keyboard device. Used in gameplay nad any visualiser scene for the game's functionality.

Note Visuals

Each Midi note from the parsed MIDI file is converted into a FallingNote. The FallingNote contains a main RectangleShape (the visible note) and three invisible trigger RectangleShapes stacked along the bottom third of the bar:

- Late trigger: the lowest third of the note bar, coloured red (255, 50, 50, 150). A keypress landing here scores a Late hit.
- Perfect trigger: the middle third, coloured green (50, 255, 50, 150). A keypress here scores a Perfect hit.
- Early trigger: the top third, coloured yellow (255, 255, 50, 150). A keypress here scores an Early hit.

Note colour changes on hit to provide immediate feedback: blue (active), yellow (early), green (perfect), red (late).

Ghost notes are only visible in Ghost Mode which overlays the previous performance's notes over the song's notes. They are a white transparent colour to show the exact timing for when they were pressed and for how long by their note duration (note height).

Scoring

We check what time the note was pressed using the collision manager to see which collision trigger was inside the piano's input trigger: late, then perfect, then early. The first matching trigger wins. If a note is pressed but no trigger is active, it counts as a Wrong note press. If a note passes below the keyboard kill trigger without being pressed it counts as a Missed note.

Triggers:

Perfect

- Score +100
- Note pressed while overlapping the perfect trigger (middle trigger). Full points.

Early

- Score +75
- Note pressed while overlapping the early trigger (bottom trigger). Partial points but it was too soon when you pressed.

Late

- Score +50
- Note pressed while overlapping the late trigger (top trigger). Partial points but it was too late when you pressed.

Miss

- Score -5

- Note not pressed and move past the input trigger and was killed by the kill trigger at the bottom of the screen. Small penalty given.

Wrong

- Score -10

- Incorrect note pressed regardless of the trigger it was occupying. Medium penalty given.

Statistics:

These note triggers record the different amount of times the above triggers were activated and is displayed in the Game Statistic's tab during gameplay. This data is then passed to the Result's scene to be displayed for the final result.

This is used for calculating the rank score that calculates:

- Timing accuracy (Early, Perfect, Late counts)
- Hit note accuracy (Perfect Notes out of Total notes, Wrong notes, Missed notes)

And gives the user a score a number that is associated with their rank (S, A, B, C, D, E)

```
void GameOver::setRank(float t_rankAmount)
{
    if (t_rankAmount >= 90.0f)
    {
        m_rankValueText.setString("S");
        m_rankValueText.setFillColor(sf::Color(255,250,0,255));
    }
    else if (t_rankAmount >= 80.0f)
    {
        m_rankValueText.setString("A");
        m_rankValueText.setFillColor(sf::Color(20, 255, 10, 255));
    }
    else if (t_rankAmount >= 60.0f)
    {
        m_rankValueText.setString("B");
        m_rankValueText.setFillColor(sf::Color(0, 20, 255, 255));
    }
    else if (t_rankAmount >= 40.0f)
    {
        m_rankValueText.setString("C");
        m_rankValueText.setFillColor(sf::Color(205, 245, 10, 255));
    }
    else if (t_rankAmount >= 20.0f)
    {
        m_rankValueText.setString("D");
        m_rankValueText.setFillColor(sf::Color(255, 125, 0, 255));
    }
    else
    {
        m_rankValueText.setString("E");
        m_rankValueText.setFillColor(sf::Color(250, 10, 30, 255));
    }
}
```

Songs

Song List:

C Scale

- Easy
- Slowest
- Single handed

Ode to Joy

- Easy
- Slow
- Single handed (but more notes than C Scale)

Major / Minor chords

- Medium
- Medium speed
- Both hands

The Scientist (Coldplay)

- Medium
- Fast speed
- Both hands (advanced chords mixed with melody notes in between)

Thousand Miles (riff only)

- Hard
- Fastest speed
- Both hands (advanced melodic notes played in isolation and in tandem with other notes, multiple octaves are used to force users to move their hand positions)

Custom Midi

Users can load their own Midi tracks that they themselves have created in a DAW or from recording during the game. They can also add Midi that they have downloaded online to be played by loading the file in the Song Selection scene.

However they cannot upload their scores or performance to the server since it isn't know in the database.

References

Official Midi Association:

<https://midi.org/>

Official Documentation:

<https://midi.org/spec-detail>

Windows

Windows Multimedia Library (Midi Input Handling, MCI Midi playback)

<https://learn.microsoft.com/en-us/windows/win32/multimedia/midi-functions>

<https://learn.microsoft.com/en-us/windows/win32/multimedia/musical-instrument-digital-interface--midi>

<https://learn.microsoft.com/en-us/windows-hardware/drivers/audio/audio-device-messages-for-midi>

Media Control Interface (Midi playback)

<https://learn.microsoft.com/en-us/windows/win32/multimedia/media-control-interface--mci>

<https://learn.microsoft.com/en-us/windows/win32/multimedia/using-the-mci-midi-sequencer>

<https://learn.microsoft.com/en-us/windows/win32/multimedia/sending-a-command>

Books:

A Beginner's Guide to Midi – R.A. Penfold
(1993 Bernard Babani LTD)

Applications:

Synthesisia:

<https://synthesiagame.com/>

Openthesia (free open-source version of Synthesia)

<https://openthesia.pages.dev/>

Repositories:

RTMidi - most popular and lightweight Midi library:

<https://github.com/thestk/rtmidi>

Mido – python Midi library:

<https://mido.readthedocs.io/en/stable/>

Midi Programming – A complete study by Stephane Richard

(Parser Explanation Articles)

<http://www.petesqbsite.com/sections/express/issue18/midifilespart1.html>

<http://www.petesqbsite.com/sections/express/issue19/midifilespart2.html>

<http://www.petesqbsite.com/sections/express/issue20/midifilespart3.html>

Videos:

Javidx9 – Midi Programming Video:

<https://youtu.be/040BKtnDdg0?si=UzKp9w7zF8tkLn6N>

Simon Hutchinson – Midi video series:

<https://www.youtube.com/@SimonHutchinson>